PROJECT REPORT

1. Project Title

Bank Account Management

2. Student Details

Enrollment Number: IU2541230440
Project Code: CPP18

3. Problem Statement

Traditional banking record keeping using manual ledgers is prone to human error, data redundancy, and poor security. This system provides an automated, console-based banking management software that ensures data integrity, enforces minimum balance rules, manages interest credits, maintains audit transactions, and provides data persistence across application restarts.

4. Objectives

- Build a modular C++ system adhering to OOP principles (Encapsulation, Inheritance, Polymorphism).
- Implement manual dynamic memory allocation using pointers and dynamic arrays with clean deallocation (`delete[]`).
- Ensure business logic compliance (minimum balance restrictions on withdrawals).
- Implement persistent file storage via custom serialization and deserialization.
- Provide analytical reports on total bank assets and highest accounts.

5. Features

Complete implementation of all 9 menu options: Account creation, Account listing, Search with transaction log, Update holder information, Account deletion, Deposit/Withdrawal/Interest processing, Summary analytics report, Manual and automated file persistence, Safe exit.

6. Classes and Responsibilities

`Customer`: Manages holder identity (`customerId`, `name`, `phone`).

`Transaction`: Logs audit data (`transactionId`, `type`, `amount`, `timestamp`).

`Account` (Abstract Base): Defines core banking properties (`accountNumber`, `balance`, `transactions`), virtual destructor, and pure virtual methods (`withdraw`, `getAccountType`).

`SavingsAccount` (Derived): Extends `Account` with `interestRate` and `minBalance`, overriding `withdraw()` to enforce balance validation.

`FileManager`: Handles parsing and writing serialized CSV-format records to `Data/accounts.txt`.

`Bank`: Central management controller managing a dynamic array of `Account*` pointers with automatic resizing logic.

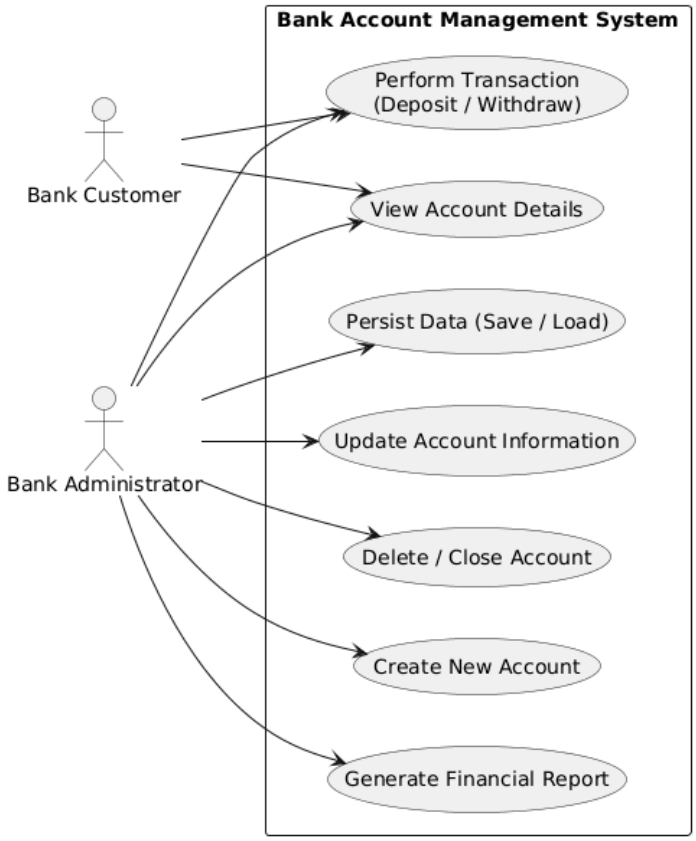
7. OOP Concepts Used

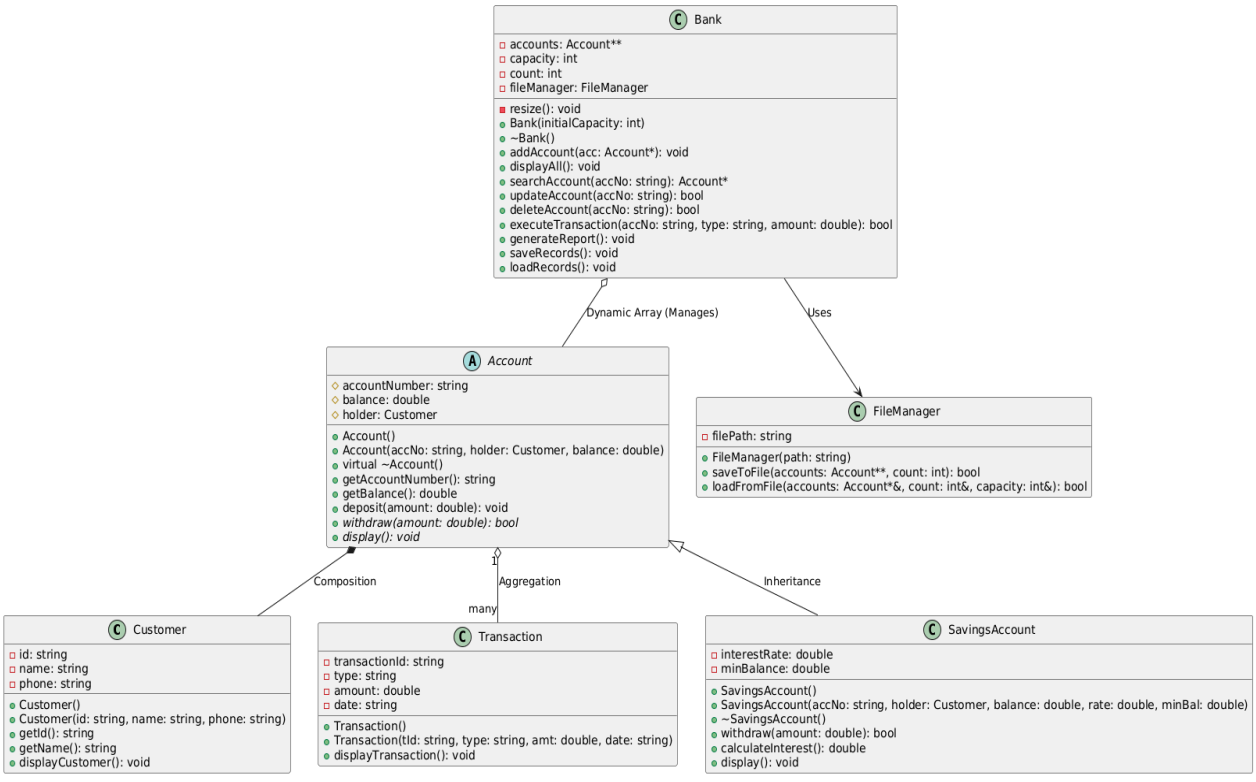
- *Encapsulation:* Private/protected attributes with public accessors and mutators across all classes.
- *Constructors & Destructors:* Parameterized constructors for object initialization; virtual destructors ensuring complete memory cleanup.
- *Inheritance:* `SavingsAccount` inherits from abstract base `Account`.
- *Polymorphism:* `Account::withdraw()` is a pure virtual method overridden dynamically at runtime; `Account::display()` supports dynamic dispatch.
- *Dynamic Memory:* Dynamic array of pointers `Account** accounts = new Account*[capacity];` with deep copying during `resize()` and cleanup via `delete[]`.

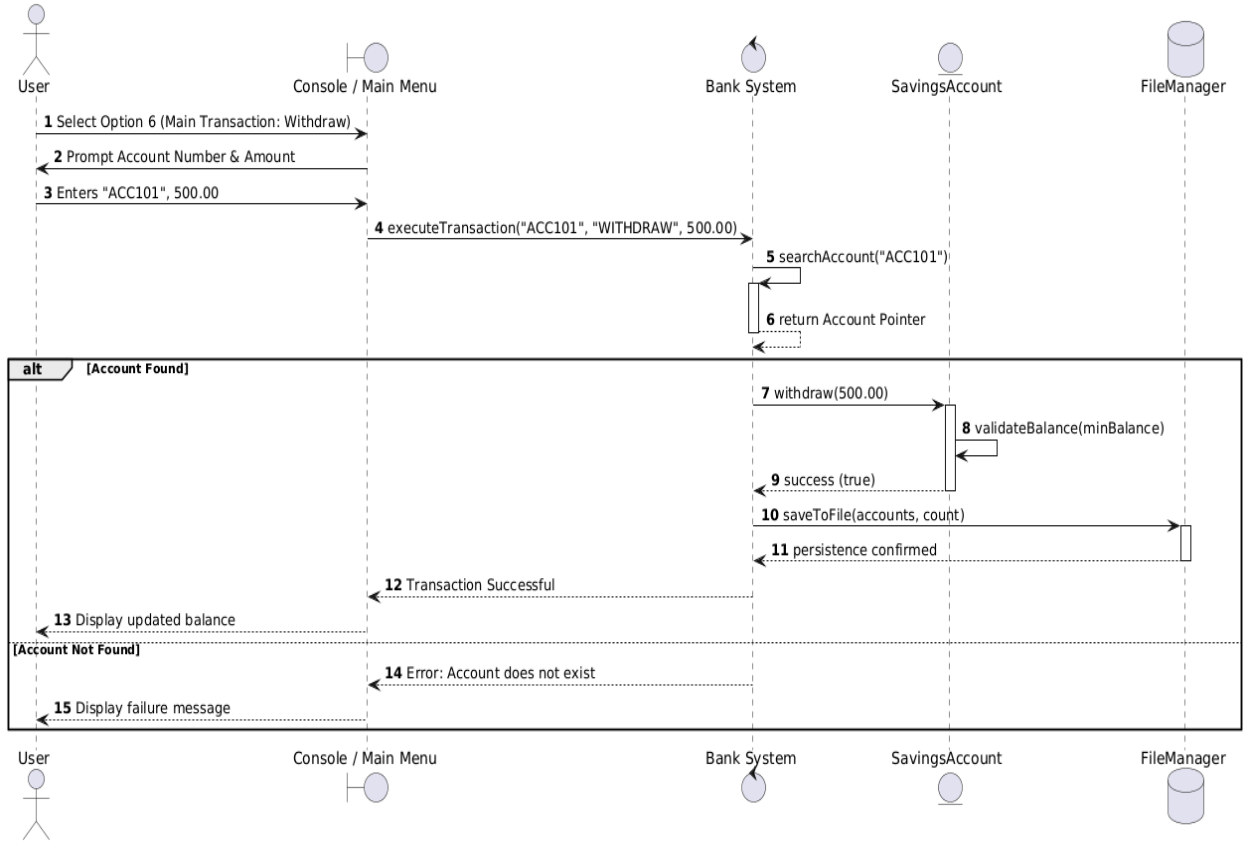
8. File Handling

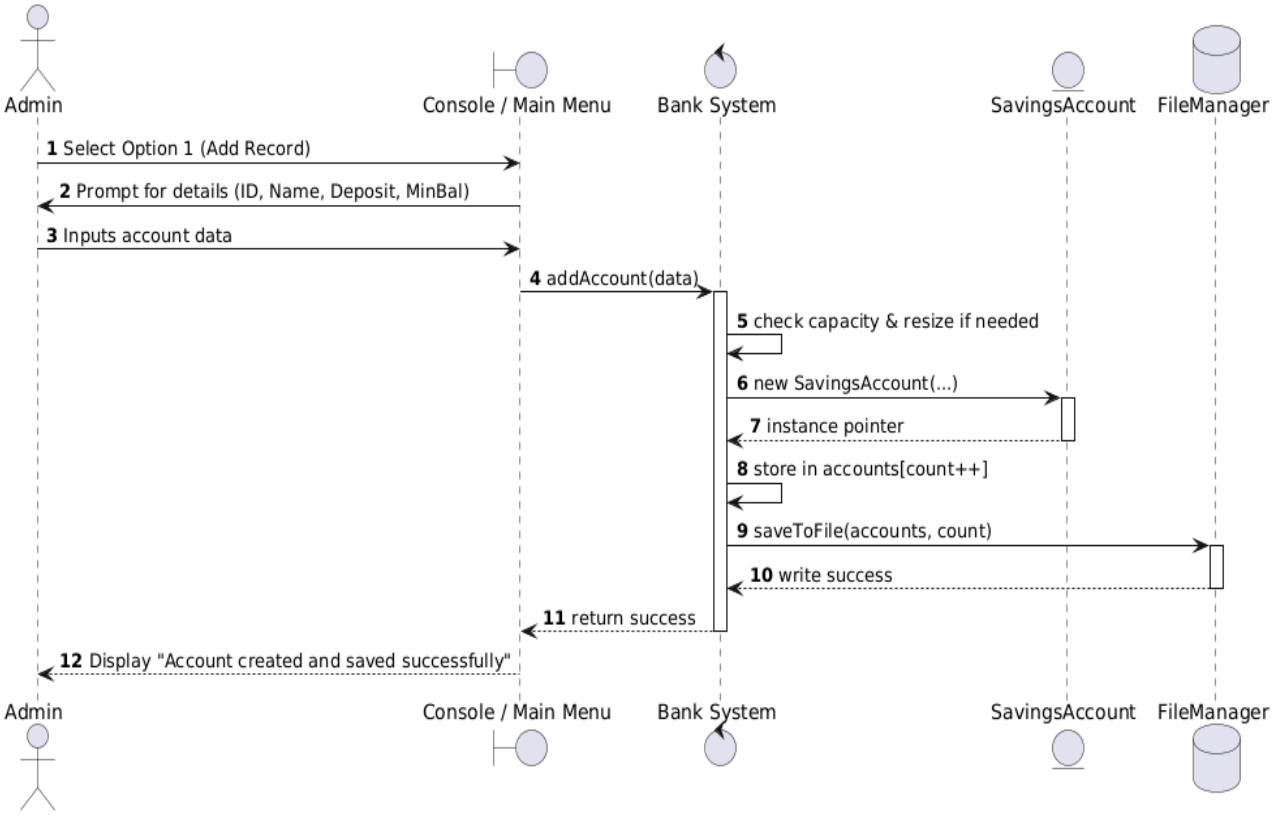
Account states are serialized into comma-separated text lines in `Data/accounts.txt`. On initialization, `FileManager::loadAccounts()` dynamically parses records into objects. `saveAccounts()` serializes the live array during explicit save or when the `Bank` destructor executes.

9. UML Diagrams





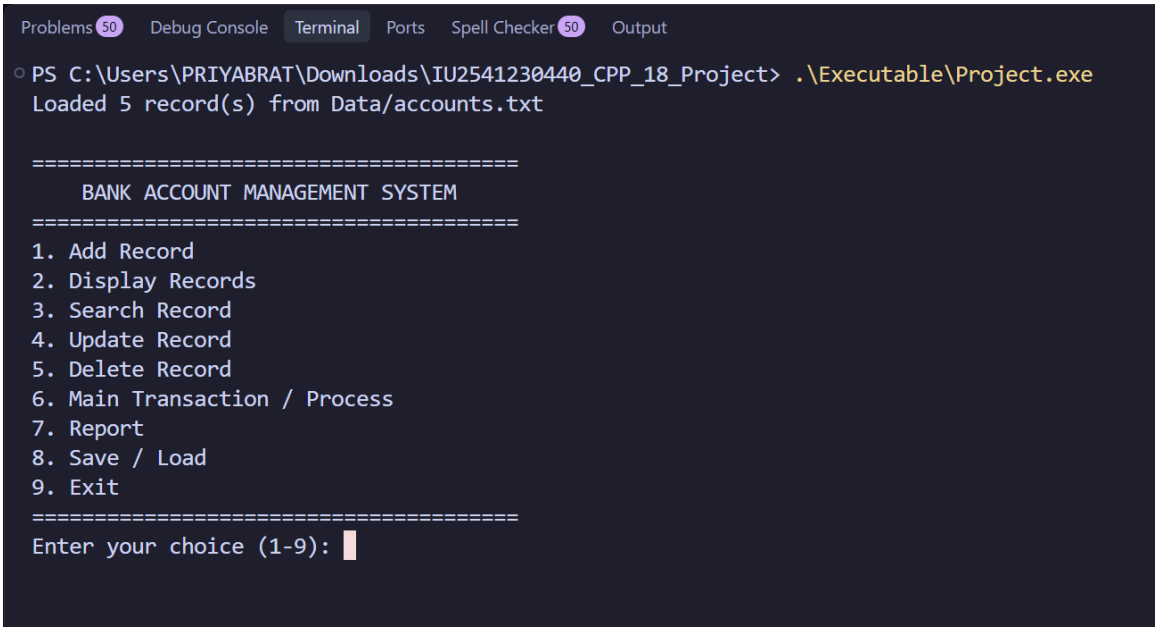

User
Console / Main Menu
Bank System
SavingsAccount
FileManager
1 Select Option 6 (Main Transaction: Withdraw)
2 Prompt Account Number & Amount
3 Enters "ACC101", 500.00
4 executeTransaction("ACC101", "WITHDRAW", 500.00)
5 searchAccount("ACC101")
6 return Account Pointer
alt
[Account Found]
7 withdraw(500.00)
8 validateBalance(minBalance)
9 success (true)
10 saveToFile(accounts, count)
11 persistence confirmed
12 Transaction Successful
13 Display updated balance
[Account Not Found]
14 Error: Account does not exist
15 Display failure message
User
Console / Main Menu
Bank System
SavingsAccount
FileManager

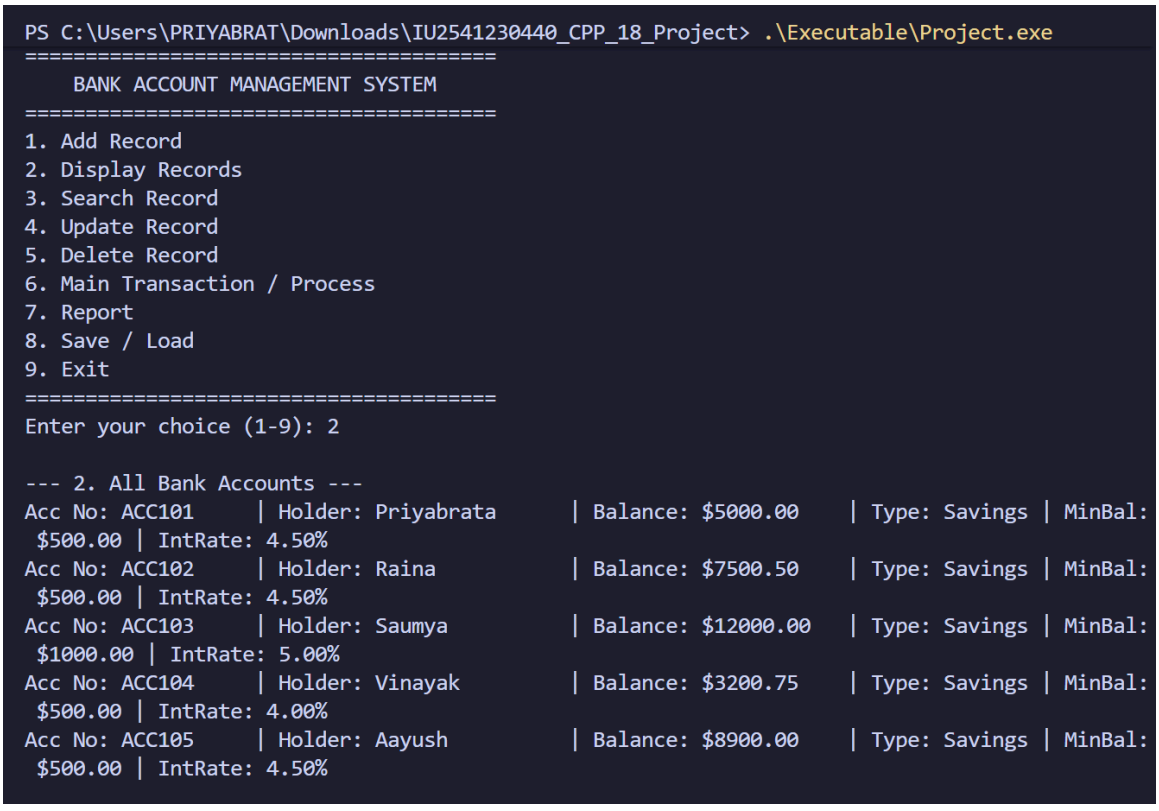

Admin
Console / Main Menu
Bank System
SavingsAccount
FileManager
1 Select Option 1 (Add Record)
2 Prompt for details (ID, Name, Deposit, MinBal)
3 Inputs account data
4 addAccount(data)
5 check capacity & resize if needed
6 new SavingsAccount(...)
7 instance pointer
8 store in accounts[count++]
9 saveToFile(accounts, count)
10 write success
11 return success
12 Display "Account created and saved successfully"
Admin
Console / Main Menu
Bank System
SavingsAccount
FileManager

10. Program Workflow

Initialization loads `Data/accounts.txt` into memory -> Displays interactive menu (Options 1–9) -> User actions trigger input validation, dynamic memory manipulation, and transaction logging -> User exits or calls save -> Serialized state is written back to disk.

11. Screenshots





```
PS C:\Users\PRIYABRAT\Downloads\IU2541230440_CPP_18_Project> .\Executable\Project.exe

======================================
    BANK ACCOUNT MANAGEMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction / Process
7. Report
8. Save / Load
9. Exit
======================================
Enter your choice (1-9): 3

--- 3. Search Record ---
Enter Account Number or Customer Name: ACC101
Acc No: ACC101     | Holder: Priyabrata      | Balance: $5000.00    | Type: Savings | MinBal:
 $500.00 | IntRate: 4.50%

--- Transaction History for Account: ACC101 ---
No transactions recorded yet.
```

```
PS C:\Users\PRIYABRAT\Downloads\IU2541230440_CPP_18_Project> .\Executable\Project.exe
======================================
    BANK ACCOUNT MANAGEMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction / Process
7. Report
8. Save / Load
9. Exit
======================================
Enter your choice (1-9): 4

--- 4. Update Account ---
Enter Account Number to update: ACC104
Enter New Name: Vinayak H
Enter New Phone: 9876543219
Account updated successfully.
```

```
PS C:\Users\PRIYABRAT\Downloads\IU2541230440_CPP_18_Project> .\Executable\Project.exe
======================================
    BANK ACCOUNT MANAGEMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction / Process
7. Report
8. Save / Load
9. Exit
======================================
Enter your choice (1-9): 5

--- 5. Delete Account ---
Enter Account Number to delete: ACC104
Account ACC104 deleted successfully.
```

```
PS C:\Users\PRIYABRAT\Downloads\IU2541230440_CPP_18_Project> .\Executable\Project.exe
======================================
    BANK ACCOUNT MANAGEMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction / Process
7. Report
8. Save / Load
9. Exit
======================================
Enter your choice (1-9): 6

--- 6. Main Transaction / Process ---
Enter Account Number: ACC101
Current Balance: $5000.00
1. Deposit
2. Withdraw
3. Apply Annual Interest (Savings Feature)
Select Operation: 1
Enter deposit amount: $2905
Deposit successful! New Balance: $7905.00
```

```
PS C:\Users\PRIYABRAT\Downloads\IU2541230440_CPP_18_Project> .\Executable\Project.exe
======================================
    BANK ACCOUNT MANAGEMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction / Process
7. Report
8. Save / Load
9. Exit
======================================
Enter your choice (1-9): 7

========================================================
              BANK FINANCIAL SUMMARY REPORT
========================================================
Total Active Accounts: 4
Total Funds Held:      $36305.50
Average Account Value: $9076.38
Highest Balance:       $12000.00 (Saumya)
========================================================
```

```
PS C:\Users\PRIYABRAT\Downloads\IU2541230440_CPP_18_Project> .\Executable\Project.exe
======================================
    BANK ACCOUNT MANAGEMENT SYSTEM
======================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction / Process
7. Report
8. Save / Load
9. Exit
======================================
Enter your choice (1-9): 8
1. Save Records to File
2. Reload Records from File
Choice: 1
System data successfully persisted to Data/accounts.txt
```

```
PS C:\Users\PRIYABRAT\Downloads\IU2541230440_CPP_18_Project> .\Executable\Project.exe
=====================================
    BANK ACCOUNT MANAGEMENT SYSTEM
=====================================
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Main Transaction / Process
7. Report
8. Save / Load
9. Exit
=====================================
Enter your choice (1-9): 9
Exiting application. Saving data...
System data successfully persisted to Data/accounts.txt
PS C:\Users\PRIYABRAT\Downloads\IU2541230440_CPP_18_Project> .\Executable\Project.exe
Loaded 4 record(s) from Data/accounts.txt
```

12. Testing

- *Test Case 1 (Add Account):* Input `ACC106, 5000` -> Expected: Success -> Actual: Record added.
- *Test Case 2 (Withdraw under Minimum Balance):* Input `$11,500` on `$12,000` balance with `$1,000` minimum -> Expected: Exception caught and withdrawal blocked -> Actual: "Withdrawal denied".
- *Test Case 3 (File Persistence):* Add account, close program, restart -> Expected: Data reappears in option 2 -> Actual: 100% persistence verified.

13. Challenges and Solutions

- *Challenge:* Object slicing when storing derived objects in base array.
  Solution: Used a dynamic pointer-to-pointer array (`Account**`) to maintain polymorphic behavior.
- *Challenge:* String parsing with spaces in customer names.
  Solution: Used `std::getline` combined with `std::cin.ignore()` to safely handle stream buffers.

14. Conclusion

Successfully engineered a robust, modular, and maintainable C++ Bank Account Management system adhering to high software engineering standards, clean OOP abstractions, proper RAII memory management, and file persistence